

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 07 -

ESCALABILIDADE AVANÇADA DE NODES COM KARPENTER

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	25
O QUE VOCÊ VIU NESTA AULA?	25
REFERÊNCIAS.....	31

EMANIP

O QUE VEM POR AÍ?

A escalabilidade de nós em clusters modernos é um desafio crescente, especialmente para lidar com picos imprevisíveis e controlar custos. O Cluster Autoscaler tradicional apresenta limitações: demora para subir nós, necessidade de muitos grupos fixos e pouca otimização de custo, especialmente em ambientes complexos com múltiplos tipos de instâncias e zonas.

Para resolver isso, a AWS criou o Karpenter, um autoscaler open source que provisiona nós sob demanda, em tempo real e sob medida para os pods pendentes, sem depender de grupos fixos. Ele é mais rápido (segundos a menos de 1 minuto para provisionar um nó), escolhe instâncias otimizadas (incluindo Spot) e elimina nós ociosos automaticamente.

Essa agilidade e eficiência tornam o Karpenter ideal para workloads variáveis e sensíveis a custo, reduzindo overhead e otimizando gastos — empresas relatam até 30% de economia.

Nesta aula, aprenderemos a usar o Karpenter na prática e exploraremos seus algoritmos e impactos técnicos, financeiros e de governança.

HANDS ON

Nesta seção, vamos colocar a mão na massa com Rust e a biblioteca kube-rs para entender, na prática, como funciona o autoscaling de nós com Karpenter. Vamos criar um mini-controlador que imita alguns comportamentos do Karpenter: detectar pods pendentes, provisionar nós sob demanda conforme os requisitos, e até consolidar nós ociosos. Tudo isso com código Rust, reforçando os conceitos enquanto construímos.



SAIBA MAIS

Detectando Pods pendentes e seus recursos

Primeiro, precisamos “ouvir” o cluster para identificar quando há pods aguardando agendamento por falta de recursos. Usando kube-rs, podemos assistir aos eventos de Pod ou listar periodicamente os pods filtrando por status. No código abaixo, conectamos ao cluster e listamos os pods pendentes, extraíndo os pedidos de CPU e memória de cada um:

```

usekube::{Client, Api, api::ListParams};
usek8s_openapi::api::core::v1::Pod;
usekube::api::ResourceExt;
usek8s_openapi::apimachinery::pkg::api::resource::Quantity

fnis_pending(pod:&Pod) ->bool {
    pod.status
        .as_ref()
        .and_then(|status|status.phase.as_deref())
        ==Some("Pending")
}

fnget_resource_requests(pod:&Pod) -> (f64, f64) {
    letmutcpu_req_millicores=0f64;
    letmutmem_req_bytes=0f64;
    ifletSome(spec) =&pod.spec {
        forcontainerin&spec.containers {
            ifletSome(resources) =&container.resources {
                ifletSome(reqs) =&resources.requests {
                    ifletSome(cpu) =reqs.get("cpu") {
                        // Parse para millicores
                        ifletOk(q) =cpu.0.parse::<Quantity>() {
                            cpu_req_millicores+=q.0asf64;
                        }
                    }
                    ifletSome(mem) =reqs.get("memory") {
                        ifletOk(q) =mem.0.parse::<Quantity>() {
                            mem_req_bytes+=q.0asf64;
                        }
                    }
                }
            }
        }
    }
    (cpu_req_millicores/1000.0, mem_req_bytes/ (1024.0*1024.0))
}

#[tokio::main]
asyncfnmain() ->anyhow::Result<()> {
    letclient=Client::try_default().await?;
    letpods_api=Api::Pod=>Api::all(client);

    letlp=ListParams::default().fields("status.phase=Pending");
    letpod_list=pods_api.list(&lp).await?;

    forpinpod_list {
        letpod_name=p.name_any();
        ifis_pending(&p) {
            println!("Pod pendente: {}", pod_name);
            let (cpu, mem) =get_resource_requests(&p);
            println!("Requer: {:.2} CPUs e {:.2} MiB memória", cpu, mem);
        }
    }
    Ok(())
}

```

Código-fonte 1– Código Rust
 Fonte: elaborado pelo autor (2025)

Usamos um field selector (status.phase=Pending) para obter apenas os pods que estão pendentes. Para cada pod, somamos as requests de CPU (em

millicores) e memória (em bytes) de seus containers. Essas informações serão essenciais para decidir que tipo de node precisamos provisionar.

GERANDO AUTOMATICAMENTE UM PROVISIONER

No Karpenter, a criação de nós é guiada por objetos customizados chamados **Provisioners** (ou NodePools, nas versões mais recentes)). Um Provisioner define regras e constraints para os nodes que o Karpenter pode lançar – como rótulos, tipos de instância permitidos, zonas, uso de Spot, entre outras políticas.

Vamos supor que nosso controlador detecte um pod pendente com requisitos específicos (por exemplo, precisa de GPU ou de arquitetura ARM). Podemos então gerar dinamicamente um Provisioner YAML adequado e criar esse recurso via API.

A seguir veremos um exemplo simplificado de um Provisioner que permite instâncias Spot em múltiplas zonas, ajustado às necessidades do pod pendente detectado:

```

usekube::api::{Api, PostParams};
useserde_json::json;
usekube::core::DynamicObject; // para CRDs desconhecidos

async fn create_provisioner_for(pod:&Pod, client:Client) -> anyhow::Result<()> {
    let prov_api:Api<DynamicObject>=Api::all(client);
    // Exemplo: escolher zona e tipo de instância com base nos labels do pod
    let zone=pod.spec.as_ref()
        .and_then(|s|s.node_selector.get("topology.kubernetes.io/zone"))
        .cloned()
        .unwrap_or("us-east-1a".to_string());
    let instance_family=if pod.spec.as_ref()
        .and_then(|s|s.annotations.get("need_gpu"))
        ==Some(&"true".to_string()) {
        "g5"// família de instâncias GPU
    } else {
        "m5"// família generalista padrão
    };
    // Monta o objeto Provisioner em JSON
    let prov=json!({
        "apiVersion":"karpenter.sh/v1alpha5",
        "kind":"Provisioner",
        "metadata": { "name":format!("prov-{}", pod.name_any()) },
        "spec": {
            "requirements": [
                { "key":"topology.kubernetes.io/zone", "operator":"In", "values": [
zone ] },
                { "key":"karpenter.sh/capacity-type", "operator":"In", "values": [
"spot", "on-demand" ] }
            ],
            "provider": {
                "instanceProfile":"KarpenterNodeInstanceProfile",
                // ... outros campos de provider (AMI, etc) conforme necessário
            },
            "limits": {
                "resources": { "cpu":1000, "memory":"2000Gi" }
            }
        }
    });
    // Envia criação do Provisioner para o cluster
    prov_api.create(&PostParams::default(), prov.as_object().unwrap()).await?;
    println!("Provisioner criado para atender pod {}", pod.name_any());
    Ok(())
}

```

Código-fonte 2 - Código Rust (2)
 Fonte: Elaborado pelo autor (2025)

Este código contrói um objeto JSON compatível com o schema do Provisioner. Adicionamos *requirements* para restringir a zona de disponibilidade (usando a zone do pod, ou um default) e permitir tanto Spot quanto On-Demand com preferência por Spot Também escolhemos dinamicamente uma família de instância (g5 se o pod precisar de GPU, senão m5).

Em cenários reais, o Provisioner incluiria muitos outros detalhes (sub-rede, AMI, rótulos, tolerações, entre outras configurações), mas aqui focamos no essencial: informar ao Karpenter **onde e como** o node deve ser criado. Após criar

desse CRD, o Karpenter real reagiria automaticamente, provisionando a VM apropriada na nuvem. **Simulando a alocação de nós otimizada por workload**

Agora vem a “mágica”: decidir **qual instância lançar e como distribuir os pods**. O Karpenter faz isso internamente avaliando os pods *unschedulable* e encontrando a opção de node mais eficiente para acomodá-los. No nosso controle simplificado, podemos **simular** esse comportamento com algumas heurísticas. Por exemplo:

- Para workloads *stateless* e tolerantes a interrupções, preferir instâncias;
- Spot: para pods críticos ou *stateful*, usar On-Demand;
- Diversificar por zona: ao lançar múltiplos nós, espalhá-los em diferentes Zonas de Disponibilidade para aumentar disponibilidade;
- Escolher o menor tipo de instância que consiga acomodar todos os pods pendentes, maximizando o uso dos recursos e evitando nós superdimensionados e ociosos.

Vamos supor que nosso cluster tenha um catálogo de tipos de instância, com seus respectivos recursos e preços. Podemos programar uma lógica de escolha.

Abaixo, mostramos uma função que, dado um conjunto de pods pendentes retorna uma lista de nós “virtuais” a lançar, decidindo Spot/On-Demand e Zonas de Disponibilidade de forma simples:

```

usestd::collections::HashMap;

structNodeSpec {
    instance_type:&'staticstr,
    cpu:u32,      // millicores
    memory:u64,   // bytes
    zone:&'staticstr,
    capacity_type:&'staticstr, // "spot" ou "on-demand"
}

letinstance_catalog:HashMap<&str, (u32, u64, f64)>=HashMap::from([
    ("t3.small", (2048, 2_u64*1024*1024*1024, 0.0104)), // 2 vCPU, 2 GiB,
    ("m5.large", (2000, 8_u64*1024*1024*1024, 0.096)),  // 2 vCPU, 8 GiB
    ("m5.xlarge", (4000, 16_u64*1024*1024*1024, 0.192)),
    ("g5.xlarge", (4800, 16_u64*1024*1024*1024, 1.0)),  // com GPU
]);

fnplan_nodes_for(pending:&[Pod]) ->Vec<NodeSpec> {
    // Agrega total de recursos pendentes
    lettotal_cpu:u32=pending.iter().map(|p|0 ).sum();
    lettotal_mem:u64=pending.iter().map(|p|0 ).sum();
    letcapacity_type=ifpending.iter().all(|p|p.annotations.get("prefer-spot")
==Some("true")) {
        "spot"
    } else {
        "on-demand"
    };
    letmutchosen_type="m5.large";
    for (&itype, (&cpu_cap, mem_cap, _price)) in&instance_catalog {
        ifcpu_cap>=total_cpu&&mem_cap>=total_mem {
            chosen_type=itype;
            break;
        }
    }
    staticAZS: [&str; 3] = ["us-east-1a", "us-east-1b", "us-east-1c"];
    usestd::sync::atomic::{AtomicUsize, Ordering};
    staticAZ_INDEX:AtomicUsize=AtomicUsize::new(0);
    letzone=AZS[AZ_INDEX.fetch_add(1, Ordering::Relaxed) %AZS.len()];
    vec![ NodeSpec {
        instance_type:chosen_type,
        cpu:instance_catalog[chosen_type].0,
        memory:instance_catalog[chosen_type].1,
        zone,
        capacity_type
    }]
}

```

Código-fonte 3 - Código Rust (3)
 Fonte: Elaborado pelo autor (2025)

Aqui, consolidamos todos os pods pendentes em um único cálculo – em um cenário real, talvez quiséssemos agrupá-los por afinidades, restrições de tolerância ou mesmo dividir a carga entre múltiplos nós, caso um único node não fosse suficiente. Mas, para simplificar, a função escolhe **um** tipo de instância que acomode todos os pods (ou o primeiro que encontrar que caiba), e assume que vamos lançar apenas um node.

Ela também decide a zona de forma round-robin para diversificar e define se será Spot ou não de acordo com uma condição (aqui, usamos uma annotation imaginária `prefer-spot: "true"` nos pods para indicar que podem ir em Spot). Essa é uma heurística simplificada, mas ilustra o tipo de decisão que o Karpenter toma: *qual a instância ótima para os pods atuais?*

Ao executar essa função com um conjunto de pods de teste, poderíamos obter resultados como:

```
let plan = plan_nodes_for(&pending_pods);
for node in plan {
    println!(
        "Planejado lançar node {} ({} CPU, {} MiB, {}) em {}",
        node.instance_type,
        node.cpu,
        node.memory / 1024 / 1024,
        node.capacity_type,
        node.zone
    );
}
```

Código-fonte 4
Fonte: elaborado pelo autor (2025)

Que poderia imprimir, por exemplo:

```
>>Planejado lançar node m5.large (2000 CPU, 8192 MiB, spot) em us-east-1a
```

Código-fonte 5
Fonte: elaborado pelo autor (2025)

Indicando um node planejado do tipo **m5.large**, Spot, na zona `us-east-1a`, com 2 vCPU e 8 GiB de RAM, suficiente para nossos pods pendentes.

MEDIÇÃO EM TEMPO REAL DOS EFEITOS DE ESCALONAMENTO

Uma vez que o node seja “lançado” (seja de fato pelo Karpenter ou simulado por nosso controlador), esperamos que os **pods pendentes** sejam rapidamente **agendados** nesse novo recurso. Podemos monitorar esse efeito medindo, por

exemplo, **o tempo de espera de um pod pendente até ser escalonado**. Em Rust, poderíamos marcar o timestamp quando detectamos o pod pela primeira vez e depois observar eventos de agendamento (pod passando a Running). Usando a API Kubernetes, podemos assistir a transição de fase:

```
use kube::runtime::{watcher, WatchStreamExt};
use futures::TryStreamExt;

let pods = Api::::all(client.clone());
let mut watch = watcher(pods, Default::default()).applied_objects().boxed();
use tokio::time::{Instant, timeout, Duration};

while let Some(Ok(pod)) = watch.try_next().await {
    if let Some(status) = &pod.status {
        if status.phase.as_deref() == Some("Running") {
            if let Some(start_time) = status.start_time.as_ref() {
                if let Some(pending_since) =
                    POD_PENDING_START.lock().unwrap().remove(&pod.name_any()) {
                    let wait = start_time.0 - pending_since;
                    println!("Pod {} agendado! Esperou {} segundos", pod.name_any(),
                        wait.as_secs());
                }
            }
        } else if status.phase.as_deref() == Some("Pending")
            && status.conditions.iter().any(|c| c.reason == Some("Unschedulable".into())) {
            // Pod sem nó disponível
            POD_PENDING_START.lock().unwrap().entry(pod.name_any()).or_insert(Instant::now());
        }
    }
}
```

Código-fonte 6

Fonte: elaborado pelo autor (2025)

No código, mantemos um registro (POD_PENDING_START) com o instante em que cada pod entrou em pendência por falta de recursos (ou seja, ausência de um node compatível). Quando o pod finalmente for executado (Running), calculamos quanto tempo passou. Isso permite medir quanto nosso autoscaler demorou para reagir.

Também podemos coletar métricas de uso de recursos dos nós (por exemplo, via metrics server ou API do K8s) antes e depois do escalonamento, para verificar se o novo node está bem utilizado. Em um cenário real, esperaríamos ver que, após acionado o Karpenter, o tempo de pendência dos pods cai significativamente, confirmando a eficiência do escalonamento em tempo real.

CONSOLIDAÇÃO CONTROLADA DE NÓS OCIOSOS

Por fim, não basta apenas escalar para cima; um autoscaler inteligente também **escala para baixo**, removendo nós desnecessários para reduzir custos. O Karpenter implementa esse processo por meio da *consolidação automática* – identifica nós subutilizados que poderiam ser eliminados sem impactar as pods (movendo essas pods para outros nós).

Vamos modelar esse comportamento: periodicamente, varrer a lista de nós no cluster, verificar aqueles com baixo uso e simular uma consolidação segura. Em pseudocódigo:

```
usekube::runtime::{watcher, WatchStreamExt};
usefutures::TryStreamExt;

letpods=Api::::all(client.clone());
letmutwatch=watcher(pods, Default::default()).applied_objects().boxed();
usetokio::time::{Instant, timeout, Duration};

whileletSome(Ok(pod)) =watch.try_next().await {
    ifletSome(status) =&pod.status {
        ifstatus.phase.as_deref() ==Some("Running") {
            ifletSome(start_time) =status.start_time.as_ref() {
                ifletSome(pending_since)
                =POD_PENDING_START.lock().unwrap().remove(&pod.name_any()) {
                    letwait=start_time.0-pending_since;
                    println!("Pod {} agendado! Esperou {} segundos", pod.name_any(),
                    wait.as_secs());
                }
            }
        } elseifstatus.phase.as_deref() ==Some("Pending")
        &&status.conditions.iter().any(|c|c.reason==Some("Unschedulable".into())) {
            // Pod sem nó disponível
            POD_PENDING_START.lock().unwrap().entry(pod.name_any()).or_insert(Instant::
            now());
        }
    }
}
```

Código-fonte 7

Fonte: elaborado pelo autor (2025)

Iteramos sobre cada Node e calculamos sua utilização (neste exemplo, considerando apenas CPU proporcional, mas poderíamos incluir memória também). Se o node estiver com menos de 20% de uso e não está vazio, ou seja, tem pods que poderiam ser movidos), tentamos validar se **esses pods** podem ser acomodados em outros nodes do cluster.

Esse é um problema de bin packing inverso: encaixar as “sobras” de um node em outros. Para simplificar assumimos que se a utilização uso é $< 20\%$, provavelmente caberá nos outros (mas em produção teríamos que garantir isso verificando espaço livre em outros nodes). Se essa condição for satisfeita, sinalizamos a consolidação: drenar os pods (respeitando PodDisruptionBudget) e remover o node.

O Karpenter realiza exatamente esse processo de forma controlada – e ainda tenta substituir nós caros por mais baratos quando possível (por exemplo, se um node On-Demand está subutilizado e poderia ser substituído por outro Spot mais barato, ele faz).

Importante: a consolidação é realizada **com muito cuidado** para não afetar aplicações – devemos honrar *graceful termination* dos pods, budgets de interrupção e espalhamento de pods, para evitar derrubar várias instâncias de um serviço de uma vez. No código acima, assumimos `can_drain` como verdade se houver capacidade sobrando alhures; em implementação real, é a parte complexa de realocar pods.

Agora que construímos um controlador simplificado em Rust:

- 1) Monitora pods pendentes,
- 2) Cria um Provisioner sob medida,
- 3) Decide qual node lançar (simulado),
- 4) Avalia o tempo de escalonamento e uso e consolida nodes ociosos.

Agora que vimos *o quê* o Karpenter faz, vamos nos aprofundar em *como* e *por que* ele o faz, com rigor técnico. Vamos analisar formalmente o problema que ele resolve, os algoritmos em jogo, comparações com o Cluster Autoscaler e questões de engenharia de larga escala.

PROBLEMA DE ESCALONAMENTO SOB RESTRIÇÕES (DEFINIÇÃO FORMAL)

O desafio de prover nós para pods pendentes pode ser formulado como um problema de **otimização combinatória sob restrições**.

Em essência, temos:

- Um conjunto de **pods** $P = \{p_1, p_2, \dots, p_n\}$ que requerem recursos (CPU, memória, etc.)

- Um conjunto de **tipos de nós** $N = \{t_1, t_2, \dots, t_m\}$ disponíveis (por exemplo, tipos de instância da nuvem, cada um com certa capacidade de CPU/memória, custo por hora, etc.).

Queremos selecionar um conjunto de nós (podem ser de tipos repetidos) para hospedar todos os pods pendentes, **minimizando um objetivo** – tipicamente, o custo total ou o número de nós – e **respeitando restrições** como capacidade do nó e afinidades.

Podemos modelar isso como um problema de **bin packing multi-dimensional**: cada pod é um item com um vetor de requisitos (por ex: (CPU, memória)), e cada node é um bin com capacidade (CPU, memórias máximas), e queremos “empacotar” todos os itens nos bins usando o menor número de bins (ou custo mínimo). Formalmente, podemos definir variáveis: x_k indicando quantos nós do tipo k lançar, e variáveis binárias $y_{i,k}$ indicando se o pod i é alocado no nó do tipo k selecionado. O problema pode ser expresso assim:

$$\begin{aligned}
 &\textbf{Minimizar: } \sum_{k \in N} cost_k \times x_k \\
 &\textbf{Sujeito a:} \\
 &\quad \sum_{k \in N} y_{i,k} = 1, \forall i \\
 &\quad \sum_{i \in P} (CPU_i \cdot y_{i,k}) \leq CPUcap_k \cdot x_k \\
 &\quad \sum_{i \in P} (Mem_i \cdot y_{i,k}) \leq MEMcap_k \cdot x_k, \forall k \\
 &\quad x \in \mathbb{Z}_{\geq 0}, y_{i,k} \in \{0,1\}
 \end{aligned}$$

Isso é, essencialmente um **modelo inteiro-linear** de bin packing multidimensional com custos (considerando CPU, memória e outros recursos).

Não por acaso, **trata-se de um problema é NP-difícil** – tanto o bin packing clássico (1 dimensão) quanto sua generalização vetorial são NP-hard e NP-completos. Em outras palavras, encontrar a alocação ótima exata em tempo polinomial é inviável para muitos pods e tipos, então precisamos recorrer a **heurísticas e aproximações**.

No Kubernetes, o scheduler enfrenta um problema análogo para colocar pods em nós já existentes (também um caso de bin packing on-line). O Karpenter lida com a variante de *adicionar novos bins* quando os atuais não bastam. Ele precisa decidir **quantos e quais nós adicionar** para acomodar pods não agendados. Dado que o espaço de possibilidades (combinações de tipos de instância, contagens e atribuições de pods) é enorme, o Karpenter usa heurísticas em vez de resolver esse ILP explicitamente.

BIN PACKING DINÂMICO E MULTIDIMENSIONAL

Uma complicação extra: o problema não é estático – os pods vêm e vão, e os nós também podem ser removidos. Temos, portanto, um cenário de **bin packing dinâmico e on-line**: novos itens (pods) chegam ao longo do tempo e devemos alocá-los sem conhecimento completo do futuro. Isso aproxima um **problema on-line** onde as decisões devem ser tomadas à medida que eventos ocorrem (não dá para esperar e agrupar tudo).

O Karpenter adota uma estratégia gulosa e reativa, baseada no estado atual: ao detectar pods pendentes, calcula rapidamente uma solução viável para acomodá-los adicionando nós. Mais tarde, se a situação muda (pods terminam, nós ficam ociosos), ele reavalia e pode consolidar. Ou seja, ele resolve repetidamente instâncias do bin packing ao longo do tempo, ajustando a alocação.

Como cada node possui múltiplos recursos (CPU, RAM, rede, talvez GPU), trata-se de um **bin packing de vetor (multidimensional)**. A eficiência de uso exige preencher bem cada dimensão sem violar nenhuma. Por exemplo, podemos ter um nó saturado em CPU mas ainda com muita memória livre – isso é fragmentação típica de multidimensional packing.

O Karpenter tenta evitar esse tipo de desperdício **escolhendo tipos de instância que correspondam ao perfil de consumo dos pods**: se os pods demandam mais memória que CPU, talvez um tipo com mais RAM seja escolhido para evitar sobra inútil; se pedem muita CPU e pouca RAM, um tipo de alta CPU é preferível. Essa correspondência fina reduz o desperdício de recursos. Em última instância, a estratégia do Karpenter é **“right-sizing”**: prover nós do tamanho certo

para os pods, ao invés de encaixar pods em nós predefinidos de tamanho fixo, como faz o Cluster Autoscaler.

Vale notar que existem algoritmos de bin packing on-line com garantias aproximadas (Next-Fit, First-Fit, Best-Fit, etc.). O Kubernetes default scheduler utiliza um estilo de *Best-Fit* inverso (prioriza espalhar ou aglomerar conforme política), enquanto um autoscaler como o Karpenter tende a fazer algo próximo de *First-Fit Decreasing*: pega os pods pendentes e tenta encaixar no menor número possível de novos nós, escolhendo instâncias grandes o suficiente porém não excessivamente maiores que o necessário (*least-waste*).

Essa é exatamente uma opção configurável no Cluster Autoscaler via “expander” – por exemplo, o expander **Least Waste** busca o node group que geraria menos sobra de recursos ao adicionar um nó. O Karpenter, por padrão, também segue a filosofia de minimizar sobra: ele calcula a combinação ótima de pods por novo node para lançar o mínimo de nodes. Internamente, o Karpenter chega a agrupar múltiplos pods num único node se possível, ao contrário do CA que às vezes adiciona um node por pod pendente mesmo que coubessem juntos.

HEURÍSTICAS: KARPENTER VS CLUSTER AUTOSCALER

O **Cluster Autoscaler (CA)** opera em ciclos (tipicamente a cada 10 segundos), em cada ciclo, ele examina os pods não agendados e simula “mentalmente” adicionar um node de cada node group para ver em qual os pods caberiam. Basicamente, o CA percorre cada pod pendente e tenta encaixá-lo nos templates de node disponíveis; se nenhum existing node group serve, ele pode falhar em escalar (é necessário ter grupos preconfigurados para cada necessidade).

O CA utiliza heurísticas configuráveis chamadas **Expanders** para decidir qual grupo escalar se múltiplos puderem atender um pod. Por exemplo:

- *Least Waste* (padrão) escolhe o node que deixará menos recursos ociosos;
- *Price* escolheria o de menor custo estimado;
- *Most Pods* o que acomoda mais pods de uma vez.

Essas heurísticas atuam no nível de grupo, e o CA adiciona 1 node por vez (ou mais, se vários pods pendentes distintos requerem grupos diferentes).

A complexidade de cada ciclo do CA cresce com o número de pods pendentes x número de grupos – e em clusters muito grandes (milhares de nodes, dezenas de grupos) isso pode se tornar pesado. De fato, o CA pode enfrentar dificuldades de performance em clusters gigantes, exigindo tunings para não ficar para trás das mudanças do cluster.

Por outro lado, em vez de depender de grupos fixos, o **Karpenter** lida com um **espaço de decisões muito maior**: pode escolher entre dezenas de tipos de instância, zonas, opções Spot vs On-Demand, etc., conforme permitido pelos Provisioners/NodePools. Isso é mais flexível, mas poderia explodir combinatoriamente. O Karpenter contorna isso aplicando filtros e ordenações, por exemplo: elimina logo tipos de instância que não atendem certos requisitos (ex: se um pod precisa de GPU, filtra só instâncias com GPU; se a zona tem que ser us-east-1a, filtra instâncias naquela zona).

Depois de aplicar esses filtros, o Karpenter **classifica as opções viáveis** com base em critérios configuráveis, como: custo, adequação de tamanho, geração recente etc. A documentação sugere práticas de limitar o mínimo possível as instâncias permitidas, para que o Karpenter tenha liberdade de otimização. Ele também **tenta alocar múltiplos pods em um mesmo novo node** se couberem, o que aumenta a utilização e reduz a contagem total de nodes.

Em termos de complexidade, o Karpenter funciona de forma reativa (pods não agendados ou avisos da nuvem) e sua tomada de decisão envolve percorrer um subconjunto de tipos disponíveis. A cada iteração, ele gera normalmente poucas instâncias (muitas vezes uma única) para resolver o backlog de pods, então o problema fica contido. Poderíamos dizer que o Karpenter sacrifica a *ótima global* em prol de decisões *gulosas rápidas* – algo desejável em ambiente altamente dinâmico.

Uma diferença importante nas heurísticas: o CA tende a preservar homogeneidade dentro de cada grupo (todos nodes de um grupo têm mesmo tipo), levando potencialmente a **sobras**. Exemplo: se um grupo tem apenas instâncias grandes, um pod pequeno aciona um node grande e deixa muita capacidade ociosa. Já o Karpenter busca o “melhor encaixe” para cada situação, podendo lançar um nó

pequeno para um pod pequeno e um nó maior quando chegam pods maiores, misturando tipos no cluster conforme a necessidade. Essa heterogeneidade controlada é precisamente como se melhora a eficiência de bin packing. Em contrapartida, a heterogeneidade pode aumentar a carga cognitiva (mais variedades de nodes para observar), mas o Karpenter gerencia isso automaticamente.

Ambos os autoscalers fazem trabalho em tempo regular. O CA, rodando a cada X segundos, pode em picos de criação de pods ficar um pouco atrás (perdendo alguns segundos preciosos até o próximo loop). O Karpenter, reagindo a eventos, pode acionar imediatamente ao surgir um pod não agendável, teoricamente ganhando alguns segundos.

Em clusters enormes, há relatos de que o Karpenter escala com menos latência perceptível que o CA. A latência de provisionar a VM em si é dominada pelo cloud provider (às vezes ~1 minuto para subir uma instância EC2 e registrá-la no cluster). O Karpenter atenua isso fazendo requisições *diretas* ao API de instâncias – se uma zona está sem capacidade, ele tenta outra quase imediatamente, enquanto o CA às vezes espera e reattempta mais devagar via ASG.

Assim, em termos de tempo *real* de escalonamento: o Karpenter costuma levar **menos de 2 minutos para entregar um node pronto**, enquanto configurações típicas de CA podem levar **vários minutos** (já vimos caso de 7 min reduzido para 2 min com Karpenter).

SIMULAÇÃO DE CUSTOS COM INSTÂNCIAS SPOT

Do ponto de vista de custos, um dos grandes trunfos do Karpenter é a sua integração com instâncias Spot (preemptíveis). Ele permite marcar pods como adequados para execução em Spot por meio de tolerações ou preferências, e assim lançar nós Spot para eles, que custam até ~70-90% menos que On-Demand.

Normalmente, instâncias Spots podem ser tiradas pelo provedor a qualquer momento com aviso curto. Uma análise de custos envolvendo Spots precisa considerar **probabilidade de interrupção vs economia**.

Podemos modelar isso com uma simulação estatística. Suponha: que uma instância On-Demand custa \$1/h e a equivalente Spot custa \$0.2/h (80% de

desconto), porém tem, digamos, 5% de chance de ser interrompida em uma hora. Em um horizonte de muitas horas, o custo esperado com Spot é muito menor, mas precisamos ter reserva para casos de interrupção (realocar pods em outra instância, possivelmente On-Demand se Spot esgotar).

Empresas como a **Tinybird** relataram economias significativas combinando Karpenter + Spot – reduziram a conta AWS em ~20% geral e até 90% em workloads de CI. O Karpenter facilita rodar o máximo possível em Spot, caindo para On-Demand só quando necessário (por falta de capacidade Spot).

Podemos modelar o custo acumulado: $CustoTotal = C_{on_demand} * Tempo_{on_demand} + C_{spot} * Tempo_{spot}$. Com o Karpenter alternando automaticamente entre esses conforme disponibilidade, a otimização vira minimizar $Tempo_{on_demand}$ sujeito a não sacrificar disponibilidade. Uma simulação de Monte Carlo poderia variar cenários de interrupção Spot e avaliar o custo vs. SLO de uptime. Em geral, se a sua aplicação tolera interrupções transientes, rodar grande parte em Spot é a estratégia de menor custo – Karpenter torna isso viável orquestrando a substituição quando necessário (ele recebe eventos de interrupção iminente e lança novos nós antecipadamente).

Além do custo por hora, outra métrica é **eficiência de utilização**: quantos recursos pagos estão realmente sendo usados por pods. Com Karpenter aumentando o bin packing, a **eficiência de CPU e memória tende a subir** (menos “buracos” ociosos em nós). Isso significa pagar menos por capacidade não utilizada. Uma estatística possível: depois de adotar Karpenter, digamos que a utilização média de CPU dos nós subiu de 40% para 60% – implicando que para a mesma carga de trabalho, agora precisamos de menos máquinas. Esse ganho se traduz diretamente em economia financeira ou em folga para rodar mais coisa no mesmo cluster.

POLÍTICAS DE INTERRUPÇÃO E QUALIDADE DE SERVIÇO (QOS)

A integração com instâncias Spot traz consigo o desafio das interrupções: quando a nuvem retira uma instância Spot (muitas vezes com aviso de 2 minutos no AWS). O Karpenter trata isso com uma **política de interrupção proativa**: ele detecta o aviso (ex.: evento do IMDS no EC2) e imediatamente marca o nó como “**terminating**”, aplicando um *taint* de interrupção e drenando os pods.

Isso dá chance dos pods reiniciarem noutro lugar antes do corte. Esse comportamento nativo simplifica garantir **consistência de QoS** – pods críticos (QoS *Guaranteed* ou com alta prioridade) podemos optar por nem colocar em Spot, mas se colocarmos, ainda assim Karpenter ajuda a mitigar downtime realocando-os. Vale lembrar que Kubernetes tem **Classes de QoS** (Guaranteed, Burstable, BestEffort) e **Pod Priority**.

Uma prioridade alta não impede interrupção Spot, mas influencia preempção e scheduling. O Karpenter atualmente não reordena pods por prioridade, porém podemos integrá-lo com Pod Priorities: por exemplo, em caso de capacidade limitada, ele atenderia todos pods pendentes (altas prioridades serão agendadas de qualquer forma primeiro pelo scheduler).

Uma coisa importante é manter **políticas de interrupção consistentes com SLOs**. Isso significa que se você tem um SLO de 99.9% uptime, não pode se dar ao luxo de ter todas réplicas de um serviço crítico em Spots vulneráveis ao mesmo tempo. Boas práticas são usar PodDisruptionBudget (PDB) para controlar quantos pods podem cair de uma vez e Topology Spread para espalhá-los em vários nodes.

O Karpenter respeita PDBs – ele não vai consolidar ou encerrar nodes se isso violar PDB (ou seja, causando interrupção acima do permitido). Assim, consegue-se consistência: por exemplo, se um banco de dados tem PDB maxUnavailable=1, o Karpenter nunca derrubará dois nós ao mesmo tempo que contenham pods dessa base.

A consistência de QoS também exige garantir que pods *Guaranteed* (geralmente os mais importantes) tenham recursos suficientes – o Karpenter depende de requests para dimensionar nós; se requests forem precisos, ele garante que esses pods rodem sem sobrecargar. Em resumo, a política é: *não sacrificar QoS ao otimizar custos*. Karpenter fornece as ferramentas (PDB, taints/tolerations, múltiplos Provisioners para separar workloads sensíveis e tolerantes) para manter cada workload no nível de confiabilidade adequado, enquanto ainda busca economias.

CONSOLIDAÇÃO SOB JANELAS MÓVEIS E TOPOLOGIAS DE PODS

A **consolidação de nós** merece uma análise mais detalhada. O Karpenter implementa a consolidação contínua, olhando o estado do cluster em janelas de tempo. Por exemplo, se um node está ocioso por X minutos, ele pode ser candidato a remoção.

Karpenter então avalia: “Se eu remover este node agora, as pods nele cabem em outros nodes existentes?”, essencialmente resolvendo um bin packing de retaguarda. Se sim, ele termina o node e deixa o scheduler redistribuir os pods (ou até o próprio Karpenter já lança um novo node menor e os pods vão pra lá). Essa decisão é repetitiva, criando um efeito de **janelas móveis**: a cada período (digamos, a cada 10 minutos) reavaliar a consolidação com dados atualizados de uso[\[19\]](#).

Isso evita remover precipitadamente um node que ficou ocioso só por 1 minuto devido a um lapso momentâneo (histerese temporal) – espera-se uma estabilidade da carga para consolidar.

A topologia dos pods alocados também entra no critério: se um node a remover contém vários pods do mesmo deployment, mas todos em outros nodes já também têm pods desse deployment, poderíamos acabar concentrando demais se não cuidarmos. Por isso, recomendam usar **Topology Spread Constraints** para distribuir pods entre AZs/nodes.

O Karpenter não implementa ativamente o spread (isso é função do scheduler), mas suas ações de consolidar podem desalocar pods; se o scheduler for livre para realocar respeitando spread, ótimo. Porém, se por azar todos pods de um serviço acabarem em nodes marcados para consolidação simultaneamente, poderia haver uma perturbação maior.

Para evitar esse cenário: o Karpenter consolida gradualmente e respeita PDB – não tira mais pods do que o budget permite. Na prática, a consolidação ocorre **um node por vez ou poucos por vez**, para o cluster absorver a realocação sem grandes sustos. A medição mostrada anteriormente (gráfico de 48h) ilustra consolidações acontecendo mesmo sem variação de carga, ou seja, Karpenter encontrando oportunidades de reduzir custos continuamente.

Matematicamente, a consolidação resolve um problema inverso: dado um conjunto de pods distribuídos em X nós, podemos reembalá-los em $X-k$ nós ($k > 0$) sem violar capacidades? Isso é NP-difícil também; Karpenter usa heurística gulosa aqui também – por exemplo, tentar mover pods do node mais vazio para outros e ver se cabe, iterativamente. Esse processo lembra algoritmos de *remoção de bin* em bin packing, que também exigem cuidado para não desemborcar em alta fragmentação.

GOVERNANÇA DO ESCALONAMENTO (RBAC, POLÍTICAS DE PRIORIZAÇÃO)

Por fim, é importante discutir como administrar e governar a autoscalabilidade em um cluster multi-tenant (múltiplos times/aplicações compartilhando). O Karpenter introduziu o conceito de **NodePools** (antes Provisioners) justamente para permitir isolamento lógico – você pode definir um NodePool por time ou por tipo de workload, cada um com suas restrições (Por exemplo., Time A só pode lançar nós até certo tamanho ou só On-Demand, Time B pode usar Spot etc.).

A governança aqui significa: **quem pode alterar as regras de autoscaling e com que limites**. Usando RBAC do Kubernetes, podemos dar acesso controlado a certos grupos para editarem seus NodePools e nada mais. Por exemplo, o time de data science poderia ter um NodePool que permite instâncias GPU, mas nenhum outro time pode “ligar” GPUs via autoscaler.

Tudo isso versionado via GitOps se desejarmos – as definições de NodePool/Provisioner podem e devem estar em repositórios Git, revisadas, aprovadas e aplicadas via pipelines (ex: ArgoCD). Essa integração GitOps garante que mudanças em política de escalonamento (como permitir um novo tipo de instância ou ajustar os limites de custos) passem por revisão e estejam rastreadas.

Políticas de priorização também são relevantes: imagine que o cluster está sob muita demanda e recursos são escassos – quais pods devem ser atendidos primeiro pelo autoscaler? O Kubernetes em si define *PriorityClass* para preempção, mas o autoscaler poderia dar preferência a escalar para pods de alta prioridade se os recursos forem limitados.

Hoje, o Karpenter trata todos os pods pendentes igualmente, mas há discussões na comunidade sobre influenciar as decisões com prioridade de pod (ex.:

se há pods baixa prioridade causando escala, talvez preferir não subir node e esperar). Por enquanto, a priorização manual vem via Partitioning: você pode dedicar um NodePool para workloads críticos (com certos labels/taints) e outro para workloads de batch.

Assim, mesmo que haja 100 pods de batch pendentes, eles não “consomem” todo o orçamento de nodes e deixam pods críticos desatendidos – cada NodePool poderia ter um **Weight** ou limite.

Embora ainda não seja um recurso nativo do Karpenter (não há peso, mas dá para setar *limits* de recursos por NodePool), é algo que empresas implementam via governança externa: monitore o gasto e limite quanto cada tipo de workload pode escalar.

Em resumo, governança de escalonamento significa **controlar o autoscaler para obedecer às políticas organizacionais**. Com Karpenter, isso é feito definindo vários Provisioners/NodePools com restrições, usando RBAC para limitar quem edita, integrando com FinOps (monitoramento de custo) para alertar se gastos excederem o previsto, e integrando com SLOs – por exemplo, não permitir escala para workloads de baixa importância se isso coloca em risco recursos para workloads de alta importância (isso poderia ser orquestrado por um controlador supervisor externo, já que nativamente o Karpenter não “sabe” da importância de cada workload, além do que você codificar nas regras dele).

Em perspectiva, podemos esperar no futuro mais inteligências nesse campo: integrar aprendizado de máquina para prever picos e pré-escalar (o que seria um “autoscaling preditivo”), e considerar atributos de workload como latência e localidade de dados nas decisões. A CNCF aponta tendências de autoscalers ficarem mais “application-aware”, entendendo requisitos específicos (como “esta carga não tolera ficar em outra região”, ou “esta aqui pode esperar alguns segundos a mais”)[50].

O Karpenter está evoluindo nessa direção – já é multicloud (suporte inicial a Azure, Alibaba Cloud e outros), o que indica um futuro onde um único autoscaler possa gerenciar recursos em diversos provedores conforme políticas de alto nível de uma organização.

MERCADO, CASES E TENDÊNCIAS

Agora vamos sair dos detalhes de implementação e observar **como o Karpenter vem sendo aplicado na prática pelas empresas**, e quais são as principais tendências de mercado.

A adoção do Karpenter vem crescendo rapidamente em organizações que operam grandes clusters Kubernetes, especialmente na nuvem pública. Aqui veremos alguns casos de uso **reais ou inspirados** em empresas conhecidas, com um olhar nos ganhos de eficiência e custo, acompanhados de visualizações para tangibilizar esses impactos.

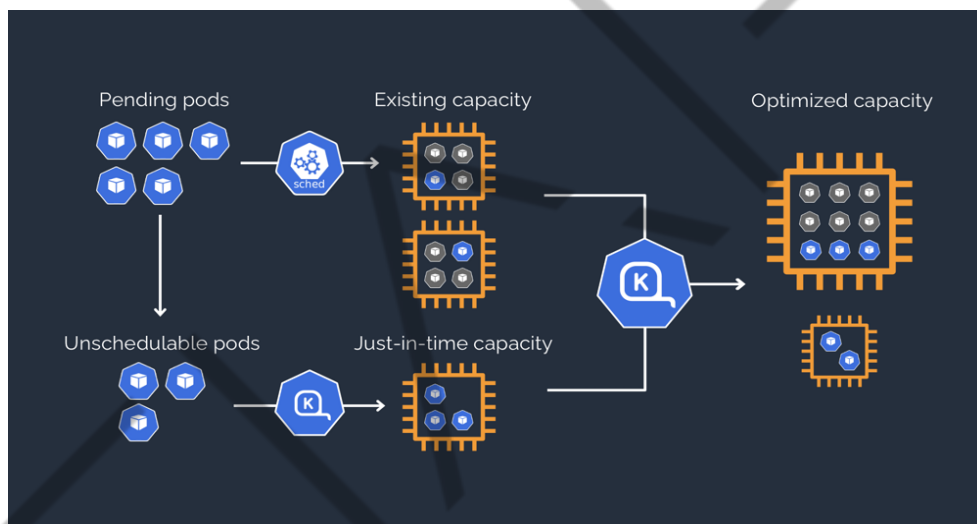


Figura 1 - Visão conceitual do funcionamento do Karpenter
Fonte: elaborado pelo autor (2025)

Pods pendentes que o scheduler nativo não alocou (unschedulable) disparam o Karpenter (símbolo K) para prover capacidade sob medida (“just-in-time capacity”). O resultado é uma capacidade otimizada, com nós adequados exatamente à necessidade atual, contrastando com a capacidade existente que poderia estar fragmentada ou insuficiente.

MERCADO LIVRE (E-COMMERCE)

A gigante latino-americana de comércio eletrônico opera um dos maiores ambientes Kubernetes da região, rodando milhares de microserviços. Eles enfrentavam o desafio de custos crescentes e complexidade com o autoscaling

tradicional. De acordo com engenheiros da Mercado Livre, a adoção do Karpenter foi motivada pela necessidade de **gerenciar custos de forma eficaz** sem sacrificar desempenho. Sendo a maior cliente AWS na América Latina, a Mercado Livre trabalhou próximo da AWS na implementação do Karpenter, colhendo benefícios de usar **instâncias mistas** e Spot para economizar. **“O Carpenter (Karpenter) nos permitiu aproveitar instâncias diversificadas e economizar dinheiro usando Spot e Reserved Instances”**, disse Marcelo Quadros, líder na empresa.

Essa flexibilidade de misturar instâncias de diferentes famílias e modelos de compra reduziu significativamente o gasto – eles mencionam que o Karpenter foi fundamental para gerir eficientemente o ambiente em expansão. Em termos operacionais, eliminaram a necessidade de manter incontáveis grupos de ASG por zona/tipo, simplificando a administração.

A cultura de plataforma da Mercado Livre também se beneficiou: os devs não precisam mais se preocupar com capacidade, o Karpenter garante que haja nós quando necessários, acoplando bem com a plataforma self-service que eles oferecem internamente. Este caso reforça que para empresas de hiper-escala, ferramentas como Karpenter não são luxo, mas necessidade para manter custos sob controle e operações enxutas.

TINYBIRD (ANALYTICS EM TEMPO REAL)

A Tinybird, startup de análise de dados em tempo real, publicou um relato detalhado de como cortou **20% dos custos AWS totais** (e até 90% em workloads CI) usando EKS + Karpenter + Spot. Eles tinham um cenário clássico: picos enormes de ingestão de dados em certos horários e ociosidade em outros. Com o Cluster Autoscaler tradicional, gerenciavam centenas de node groups para cobrir todos tipos e zonas, o que era difícil de manter.

Após migrarem para o Karpenter, conseguiram consolidar tudo em **um único nodepool unificado**, separando logicamente apenas em dois: um NodePool para workloads críticos/estaduais (que usam On-Demand) e um para workloads stateless (100% Spot). Isso simplificou demais a operação e potencializou a economia com Spot. No gráfico abaixo, tirado de um dos clusters da Tinybird, vemos a atividade do

Karpenter em 48 horas, incluindo **provisionamento e término de nós continuamente**:



Figura 2 - Atividade de provisionamento e consolidação de nós em um cluster (case Tinybird) ao longo de 48h
Fonte: (2025)

Linhas para cima representam nós sendo criados (em verde/azul, diferenciando NodePools), e linhas para baixo representam nós sendo terminados (roxo/magenta). Mesmo sem grandes variações de carga, o Karpenter identifica nós subutilizados e os remove (“consolidation”), otimizando custos em tempo real. Observa-se um padrão cíclico de ajustes finos – isso é o autoscaler trabalhando 24/7 como um engenheiro de plataforma incansável.

Nesse cluster, a Tinybird configurou o NodePool *Critical* para sempre ter instâncias On-Demand suficientes para cargas críticas (cobertas por Savings Plans da AWS) e um NodePool *Stateless* só com Spots para microserviços tolerantes a falhas. O gráfico mostra que, apesar da carga constante, o Karpenter fez várias ações de criar/terminar nós – ou seja, consolidou: removeu nós que ficavam pouco usados e provisionou outros onde precisava (às vezes substituindo On-Demand por Spot quando possível).

Eles relataram que, com isso, conseguiram sustentar **picos de ingestão de bilhões de registros** sem suar, pagando bem menos. Importante também foram as práticas acompanhando o Karpenter: implementação de *graceful shutdown* nos serviços, uso de PDB para evitar retirar muitos pods de uma vez, e *topology spread* para resiliência – tudo para assegurar que as otimizações do Karpenter não causariam instabilidade. Em suma, a Tinybird ilustra a **sinergia entre Karpenter e arquitetura otimizada para autoscaling**: eles ajustaram suas aplicações e configs (HPA com KEDA para escalar pods rapidamente, Karpenter para nós, e observabilidade em tempo real) e colheram grandes ganhos de eficiência.

PRODIGY EDUCATION (EDTECH)

A Prodigy, plataforma de jogos educacionais, apresentou um case via AWS mostrando como **escalar de 34 para 430 nós** automaticamente para atender 200 mil usuários simultâneos, graças ao Karpenter. Antes, eles superprovisionavam recursos para garantir performance em picos, o que gerava muito custo ocioso. Com EKS + Karpenter, passaram a escalar somente quando precisava e de forma muito mais ágil – em teste, viram o tempo de scale-up cair de ~7 minutos (solução antiga) para ~2 minutos com Karpenter.

Isso significa que um spike matinal de usuários agora é atendido **5 minutos mais rápido**, melhorando a experiência (menos filas ou lentidão no jogo) e permitindo reagir a aumentos súbitos sem quedas. O resultado financeiro também foi impressionante: a Prodigy reportou **40–60% de economia em custos de compute** após otimizar com o Karpenter. Isso sem dúvida envolve altíssima utilização de Spot – eles citam que a maioria dos clusters roda em Spot e que nunca chegaram a ficar sem capacidade Spot para o NodePool stateless deles.

Além disso, o Karpenter trouxe **autonomia aos desenvolvedores**: antes, cada ajuste de capacidade dependia do time de infra; agora, os devs podem lançar serviços e confiar que o autoscaler trará os recursos necessários. Essa independência é algo buscado em práticas de *Platform Engineering*: Karpenter, configurado corretamente, atua como um “resolvedor universal” de necessidades de compute para os devs, dentro dos limites definidos pela plataforma.

ADOBE E CNCF

Em termos de tendência de mercado, vale citar que empresas como a Adobe não só usam o Karpenter, mas **influenciam ativamente seu futuro**. A Adobe foi um dos patrocinadores para levar o Karpenter para incubação no CNCF, colaborando para torná-lo mais multicloud (inclusão de Azure, etc.).

Isso mostra confiança de grandes players na ferramenta e interesse em padronizá-la como peça da stack cloud native. O fato da Adobe ter trazido a Microsoft para contribuir indica que vemos um movimento de **diversificação**: embora nascido na AWS, o Karpenter caminha para servir de autoscaler também no Azure Kubernetes

Service (AKS) e possivelmente outras nuvens. No estado atual (2025), AWS e Azure já são suportados, e a comunidade fala sobre GCP no horizonte. Essa convergência sugere que o Karpenter pode virar “o” autoscaler padrão em muitos lugares, substituindo ou complementando o Cluster Autoscaler tradicional.

TENDÊNCIAS TECNOLÓGICAS

Além dos casos concretos, é interessante notar algumas tendências emergentes envolvendo o Karpenter: - **Integração com GitOps**: Times estão incorporando as configs do Karpenter em fluxos GitOps para controle de versões e mudanças auditáveis. Por exemplo, a nOps (plataforma de otimização) anunciou suporte GitOps onde você gerencia Karpenter tanto via UI quanto via código fonte.

Isso significa que aprovar uma nova família de instância permitida pode ser um *pull request* revisado, em vez de um `kubectl` manual em produção. Essa tendência aumenta confiança na automação – trata autoscaling como código, com testes e *review*. - **FinOps e observabilidade de custos**: Ferramentas de FinOps (como OpenCost, Kubecost) estão sendo usadas junto com o Karpenter para monitorar em tempo real quanto se está gastando e economizando. Por exemplo, relatórios que mostram “sua economia hoje por usar Spots foi X” ou “você poderia economizar Y consolidando mais”.

O Karpenter fornece métricas e eventos (por exemplo, evento de node consolidado com reason “underutilized”) que podem alimentar dashboards de custo. A união de **FinOps + Karpenter** tende a automatizar ainda mais a otimização: algumas empresas podem colocar limites – ex: se custo/hora exceder certo valor, reduzir alguns workloads de prioridade menor. O Karpenter já dá a base para isso ao permitir *limits* globais de recursos (você pode limitar por exemplo máximo de 1000 CPUs no cluster), servindo de salvaguarda financeira.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, entendemos os desafios de escalar nós em Kubernetes, especialmente com workloads dinâmicos e custos a controlar. Vimos que o Cluster Autoscaler tradicional tem limitações, o que motivou a criação do Karpenter.

O Karpenter oferece uma escalabilidade mais ágil e flexível, reagindo a eventos em tempo real e escolhendo tipos de instância sob medida (incluindo Spot) para otimizar custo e eficiência. Ele usa heurísticas para lidar com o problema complexo de bin packing e ainda consolida nós ociosos, respeitando regras de disponibilidade (PDBs, spread).

Casos reais mostram economias significativas e ganhos operacionais. Também discutimos como integrar governança (RBAC, políticas), GitOps e FinOps para controle e segurança no escalonamento.

Por fim, o Karpenter representa uma evolução cultural em Kubernetes, promovendo automação inteligente, eficiência e práticas modernas de infraestrutura.

REFERÊNCIAS

AWS. **Saving Up to 60% on Compute Costs Using Amazon EKS and Karpenter with Prodigy Education.** Amazon Web Services – Customer Stories. 2024. Disponível em: <https://aws.amazon.com/solutions/case-studies/prodigy-education-eks-case-study/>. Acesso em: 9 set. 2025.

DUGGAL, T. **Karpenter vs Cluster Autoscaler: The Ultimate Guide.** PerfectScale Blog. 2024. Disponível em: <https://www.perfectscale.io/blog/karpenter-vs-cluster-autoscaler>. Acesso em: 9 set. 2025.

KARPENTER DOCS. **Concepts & Features.** Karpenter.sh. 2023. Disponível em: <https://karpenter.sh/>. Acesso em: 9 set. 2025.

KOMODOR (ADOBE). **Adobe End User Journey – CNCF Report.** 2023. Disponível em: <https://www.cncf.io/reports/adobe-end-user-journey-report/>. Acesso em: 9 set. 2025.

KOMODOR (MERCADO LIBRE). **Kubernetes for Humans Podcast #016 – Transcript.** Komodor Resources. 2023. Disponível em: <https://komodor.com/resources/016-kubernetes-for-humans-podcast-with-marcelo-quadros-juliano-martins-mercado-libre/>. Acesso em: 9 set. 2025.

KUBERNETES. **Resource Bin Packing – Bin Packing in Kubernetes.** Kubernetes.io. [s.d.]. Disponível em: <https://kubernetes.io/docs/concepts/scheduling-eviction/resource-bin-packing/>. Acesso em: 9 set. 2025.

MOUSTAKIS, I. **Karpenter vs. Cluster Autoscaler – Kubernetes Scaling Tools.** Spacelift Blog. 2024. Disponível em: <https://spacelift.io/blog/karpenter-vs-cluster-autoscaler>. Acesso em: 9 set. 2025.

RODRÍGUEZ, L. **We cut AWS costs by 20% while scaling faster with EKS, Karpenter, and Spot Instances.** Tinybird Blog. 2025. Disponível em: <https://www.tinybird.co/blog-posts/how-we-cut-aws-costs-while-scaling-faster-with-eks-karpenter-and-spot-instances>. Acesso em: 9 set. 2025.

SHADOW-SOFT. **Kubernetes Scheduler Algorithm Combinatorial Modeling.** 2021. Disponível em: <https://shadow-soft.com/content/kubernetes-scheduler-algorithm-combinatorial-modeling>. Acesso em: 9 set. 2025.

SIG AUTOSCALING GITHUB. **Karpenter AWS – README.** GitHub. 2023. Disponível em: <https://github.com/aws/karpenter/>. Acesso em: 9 set. 2025.

PALAVRAS-CHAVE

Kubernetes. Karpenter. Autoscaler.

EMSE



POS TECH